Experiment 1: Analyze Tokenization Techniques and Embedding Representations in Large Language Models.

Aim: To analyze different tokenization techniques and study embedding representations in Large Language Models (LLMs) using practical implementation and visualization.

Procedure:

Install required libraries
Install transformers, torch, matplotlib, and scikit-learn.

Load tokenizers and tokenize text
Use BERT and GPT-2 tokenizers to split input text into tokens (WordPiece and BPE).

Convert tokens to numerical IDs
Transform tokens into input IDs that can be processed by the model.

Load pre-trained model and pass inputs
Use a model like BERT to process token IDs and generate outputs.

Extract embeddings
Obtain token embeddings from the model (last hidden state, shape: [1, seq_len, 768]).

Visualize embeddings
Apply PCA to reduce dimensions and plot embeddings in 2D for visualization.

Code :

```
from transformers import AutoTokenizer, AutoModel

import torch

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

text = "Large Language Models are transforming artificial intelligence."

bert_tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")

gpt_tokenizer = AutoTokenizer.from_pretrained("gpt2")

print("\nBERT Tokens (WordPiece):")

bert_tokens = bert_tokenizer.tokenize(text)

print(bert_tokens)
```

```
print("\nGPT-2 Tokens (BPE):")
gpt_tokens = gpt_tokenizer.tokenize(text)
print(gpt_tokens)
bert_inputs = bert_tokenizer(text, return_tensors="pt")
print(bert_inputs["input_ids"])
model = AutoModel.from_pretrained("bert-base-uncased")
with torch.no_grad():
    outputs = model(**bert_inputs)
embeddings = outputs.last_hidden_state
print(embeddings.shape)   # Expected: [1, seq_len, 768]
embedding_np = embeddings[0].numpy()
pca = PCA(n_components=2)
reduced = pca.fit_transform(embedding_np)
tokens = bert_tokenizer.convert_ids_to_tokens(bert_inputs["input_ids"][0])
plt.figure()
plt.scatter(reduced[:, 0], reduced[:, 1])
for i, token in enumerate(tokens):
    plt.annotate(token, (reduced[i, 0], reduced[i, 1]))
plt.title("2D Visualization of Token Embeddings (PCA)")
plt.xlabel("Component 1")
plt.ylabel("Component 2")
plt.grid()
plt.show()
print("\nEmbedding Shape:", embeddings.shape)
```

Output:

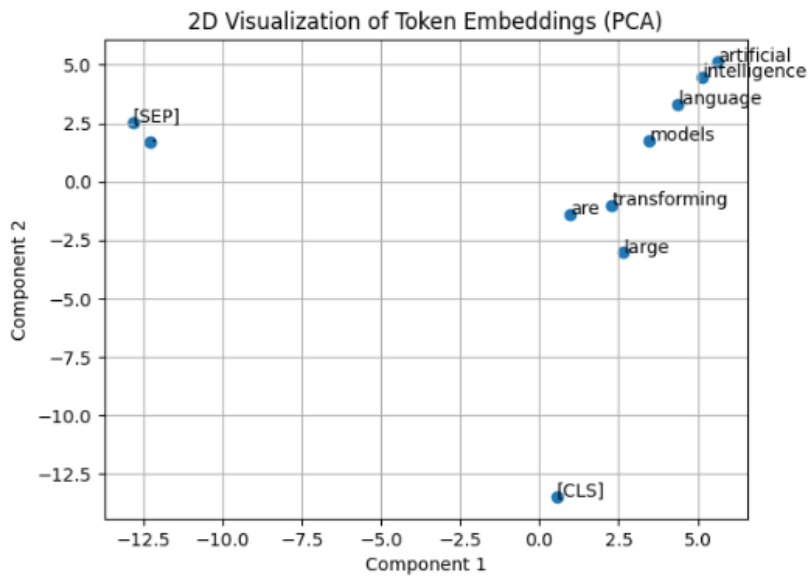
```
BERT Tokens (WordPiece):
['large', 'language', 'models', 'are', 'transforming', 'artificial', 'intelligence', '.']

GPT-2 Tokens (BPE):
['Large', 'ĠLanguage', 'ĠModels', 'Ġare', 'Ġtransforming', 'Ġartificial', 'Ġintelligence', '.']

===== TOKEN IDs =====
tensor([[  101,  2312,  2653,  4275,  2024, 17903,  7976,  4454,  1012,   102]])
Loading weights: 100% 199/199 [00:01<00:00, 144.90it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: bert-base-uncased
Key                                          | Status     |  |
---------------------------------------------+------------+--+-
cls.predictions.transform.dense.bias         | UNEXPECTED |  |
cls.seq_relationship.bias                    | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.bias     | UNEXPECTED |  |
cls.predictions.transform.LayerNorm.weight   | UNEXPECTED |  |
cls.predictions.bias                         | UNEXPECTED |  |
cls.predictions.transform.dense.weight       | UNEXPECTED |  |
cls.seq_relationship.weight                  | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

===== EMBEDDING SHAPE =====
torch.Size([1, 10, 768])
```




```
Embedding Shape: torch.Size([1, 10, 768])
```

Result:

Tokenization techniques and embedding representations in Large Language Models were successfully analyzed. The experiment demonstrated how different tokenization strategies affect text processing and how contextual embeddings capture semantic meaning in high-dimensional space. The relationship between tokens and their numerical representations was clearly observed and visualized

Experiment 2: Implement and Visualize Self-Attention and Multi-Head Attention Mechanisms

Aim

To implement scaled dot-product self-attention and multi-head attention mechanisms and visualize their attention weight distributions.

Procedure:

1. Install and import required libraries
   Install and import PyTorch, NumPy, Matplotlib, and Seaborn for computation and visualization.
2. Implement scaled dot-product attention
   Define a function to compute attention scores using Query (Q), Key (K), and Value (V), apply scaling and softmax to obtain attention weights.
3. Create sample input tensors
   Initialize random tensors for Q, K, and V representing token embeddings.
4. Compute self-attention output
   Pass Q, K, V into the attention function to obtain output vectors and attention weight matrix.
5. Visualize attention weights
   Plot a heatmap of the attention matrix to observe how tokens attend to each other.
6. Implement multi-head attention
   Split Q, K, V into multiple heads, apply attention in parallel, and concatenate outputs to obtain final representation.

Code:

```
import torch

import torch.nn.functional as F

import matplotlib.pyplot as plt

import seaborn as sns

def scaled_dot_product_attention(Q, K, V):

    d_k = Q.size(-1)
```

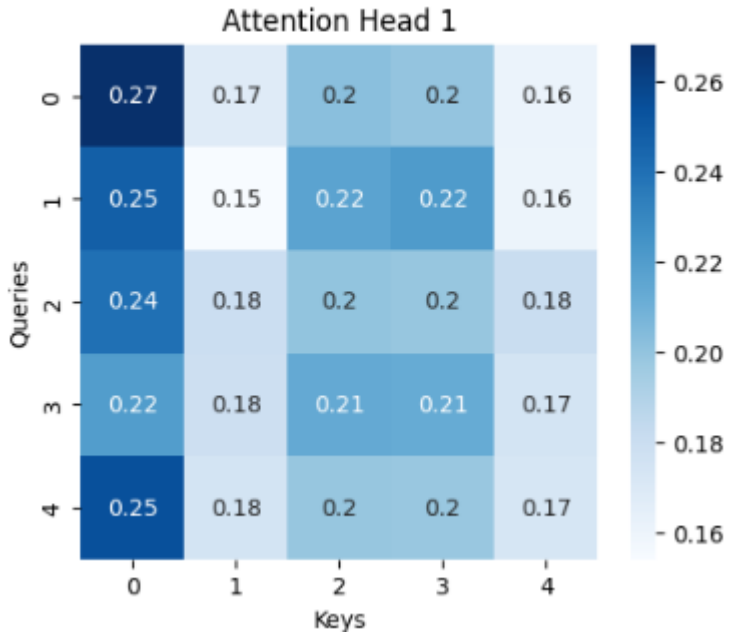
```
    scores = torch.matmul(Q, K.transpose(-2, -1)) / torch.sqrt(torch.tensor(d_k,
dtype=torch.float32))
    output = torch.matmul(attention_weights, V)
    return output, attention_weights
torch.manual_seed(42)
Q = torch.rand(1, 5, 8)   # (batch_size, seq_len, embed_dim)
K = torch.rand(1, 5, 8)
V = torch.rand(1, 5, 8)
output, attention_weights = scaled_dot_product_attention(Q, K, V)
print("Output Shape:", output.shape)          # [1, 5, 8]
print("Attention Shape:", attention_weights.shape)  # [1, 5, 5]
plt.figure(figsize=(6,5))
sns.heatmap(attention_weights[0].detach().numpy(),
            annot=True,
            cmap="Blues")
plt.title("Self-Attention Weight Matrix")
plt.xlabel("Keys")
plt.ylabel("Queries")
plt.show()
def multi_head_attention(Q, K, V, num_heads=2):
    batch_size, seq_len, embed_dim = Q.size()
    head_dim = embed_dim // num_heads
    Q = Q.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
    K = K.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
    V = V.view(batch_size, seq_len, num_heads, head_dim).transpose(1, 2)
    outputs = []
    weights_all = []
    for i in range(num_heads):
```

```
        out, weights = scaled_dot_product_attention(Q[:, i], K[:, i], V[:, i])
        outputs.append(out)
        weights_all.append(weights)
    concat_output = torch.cat(outputs, dim=-1)
    return concat_output, weights_all
multi_output, multi_weights = multi_head_attention(Q, K, V)
for i, weights in enumerate(multi_weights):
    plt.figure(figsize=(5,4)
    sns.heatmap(weights[0].detach().numpy(),
                annot=True,
                cmap="Blues")
    plt.title(f"Attention Head {i+1}")
    plt.xlabel("Keys")
    plt.ylabel("Queries")
    plt.show()
```

Output:

```
Output Shape: torch.Size([1, 5, 8])
Attention Shape: torch.Size([1, 5, 5])
```

```
===== MULTI-HEAD ATTENTION =====
Multi-Head Output Shape: torch.Size([1, 5, 8])
```



Result: Self-attention captures contextual importance among tokens using attention weights. Multi-head attention enhances learning by modeling multiple relationships in transformer-based LLMs.

Experiment 3: Apply Pretrained Language Models for NLP Tasks

Aim:
To apply pretrained transformer-based language models to perform various Natural Language Processing (NLP) tasks such as text classification, named entity recognition, question answering, and
text summarization.

Procedure:

Install and import required libraries
Install Hugging Face Transformers, PyTorch, and Datasets library.

Load pretrained pipelines
Use Hugging Face pipeline() API to load models for different NLP tasks.

Perform text classification
Apply sentiment analysis pipeline to classify input text as positive or negative.

Perform named entity recognition (NER)
Identify entities like person, organization, and location from text.

Perform question answering
Provide context and question to extract relevant answer from the text.

Perform text summarization
Generate a concise summary from a longer input text.

Code:

```
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

text1 = "Large Language Models are revolutionizing AI research."

result1 = classifier(text1)

print("Input:", text1)

print("Output:", result1)

ner = pipeline("ner", grouped_entities=True)

text2 = "Elon Musk founded SpaceX in the United States."
```

```
result2 = ner(text2)

print("Input:", text2)

print("Output:", result2)

qa = pipeline("question-answering")

context = "Transformers are deep learning models introduced in 2017."

question = "When were Transformers introduced?"

result3 = qa(question=question, context=context)

print("Question:", question)

print("Answer:", result3)

text3 = """Large Language Models are trained on massive datasets

and can perform multiple NLP tasks effectively.

They are widely used in modern AI applications."""

summary = summarizer(text3, max_length=30, min_length=10, do_sample=False)

print("Original Text:", text3)

print("Summary:", summary)
```

Output:

```
===== SENTIMENT ANALYSIS =====
config.json: 100% 629/629 [00:00<00:00, 16.1kB/s]
model.safetensors: 100% 268M/268M [00:05<00:00, 50.7MB/s]
Loading weights: 100% 104/104 [00:00<00:00, 399.92it/s, Materializing param=pre_classifier.weight]
tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 1.70kB/s]
vocab.txt: 232k/? [00:00<00:00, 6.16MB/s]
Input: Large Language Models are revolutionizing AI research.
Output: [{'label': 'POSITIVE', 'score': 0.9985295534133911}]

===== NAMED ENTITY RECOGNITION =====
Loading weights: 100% 391/391 [00:00<00:00, 692.76it/s, Materializing param=classifier.weight]
BertForTokenClassification LOAD REPORT from: dbmdz/bert-large-cased-finetuned-conll03-english
Key                        | Status     |  |
---------------------------+------------+--+-
bert.pooler.dense.bias     | UNEXPECTED |  |
bert.pooler.dense.weight   | UNEXPECTED |  |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
Input: Elon Musk founded SpaceX in the United States.
Output: [{'entity': 'I-PER', 'score': np.float32(0.9994566), 'index': 1, 'word': 'El', 'start': 0, 'end': 2}, {'entity': 'I-PER', 'score': n

===== QUESTION ANSWERING =====
config.json: 100% 473/473 [00:00<00:00, 45.2kB/s]
model.safetensors: 100% 261M/261M [00:03<00:00, 148MB/s]
Loading weights: 100% 102/102 [00:00<00:00, 357.83it/s, Materializing param=qa_outputs.weight]
tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 4.77kB/s]
vocab.txt: 213k/? [00:00<00:00, 5.66MB/s]
tokenizer.json: 436k/? [00:00<00:00, 13.4MB/s]
```

Result:

Pretrained language models were applied to NLP tasks like sentiment analysis, NER, QA, and summarization using transfer learning.
The experiment showed how contextual embeddings enable efficient problem solving without training models from scratch.

Experiment 4: Design and Evaluate Prompting Strategies for In-Context Learning

Aim
To design different prompting strategies and evaluate their effectiveness in performing tasks using in-context learning with Large Language Models (LLMs).

Procedure:

1. Install and import required libraries
   Install Transformers and PyTorch, and load a pretrained text-generation model.
2. Apply zero-shot prompting
   Provide only task instruction without examples and generate output.
3. Apply one-shot prompting
   Provide one example along with the task to guide the model.
4. Apply few-shot prompting
   Provide multiple examples to improve accuracy and structure of output.
5. Apply chain-of-thought prompting
   Include step-by-step reasoning instructions to improve logical responses.
6. Apply role-based prompting
   Assign a role/persona to the model to influence tone and style of output.

Code:

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

print("===== ZERO-SHOT PROMPTING =====")

prompt = "Classify the sentiment of this sentence as Positive or Negative:\nThe movie was fantastic and inspiring."

result = generator(prompt, max_length=50)

print(result[0]['generated_text'])
```

```
print("\n===== ONE-SHOT PROMPTING =====")

prompt = """Classify sentiment:

Sentence: I love this product.

Sentiment: Positive

Sentence: The service was terrible.

Sentiment:"""

result = generator(prompt, max_length=60)

print(result[0]['generated_text'])

print("\n===== FEW-SHOT PROMPTING =====")

prompt = """Classify sentiment:

Sentence: I love this product.

Sentiment: Positive

Sentence: The service was terrible.

Sentiment: Negative

Sentence: The food was amazing.

Sentiment:"""

result = generator(prompt, max_length=80)

print(result[0]['generated_text'])

print("\n===== CHAIN-OF-THOUGHT PROMPTING =====")

prompt = """Solve step by step:

If a shop sells 5 books for $50, what is the price of one book?

Explain your reasoning."""

result = generator(prompt, max_length=100)

print(result[0]['generated_text'])

print("\n===== ROLE-BASED PROMPTING =====")

prompt = "You are a professional financial advisor. Explain the importance of saving money."

result = generator(prompt, max_length=100)
```

print(result[0]['generated_text'])

Output:

```
===== ONE-SHOT PROMPTING =====
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=80) seem to have been set. `max_new_tokens` will take precedence.
Classify sentiment:
Sentence: I love this product.
Sentiment: Positive
Sentence: The service was terrible.
Sentiment: Negative
Sentence: Overall very good service.
Sentiment: Very good service.
Sentiment: Good service.
Sentiment: Very good service.
Sentiment: Good service.
Sentiment: Good service.
Sentiment: Good service.
Sentiment: Good service.
Sentiment: Good service.
Sentiment: Good service.
Sentiment: Good service.
Sentiment: Good service.
```

```
===== FEW-SHOT PROMPTING =====
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=100) seem to have been set. `max_new_tokens` will take precedence
Classify sentiment:
Sentence: I love this product.
Sentiment: Positive
Sentence: The service was terrible.
Sentiment: Negative
Sentence: The food was amazing.
Sentiment: Positive
Sentence: The customer service was great.
Sentiment: Negative
Sentence: The service was terrible.
Sentiment: Positive
Sentence: The customer service was great.
Sentiment: Negative
Sentence: The service was terrible.
Sentiment: Positive
```

```
sent

===== CHAIN-OF-THOUGHT PROMPTING =====
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Both `max_new_tokens` (=256) and `max_length`(=100) seem to have been set. `max_new_tokens` will take precedence. Please
Solve step by step:
If a shop sells 5 books for $50, what is the price of one book?
Explain your reasoning. If someone tells you they purchased 5 books for $50 you don't understand them.
If you read five books and it costs $50, how do you know that it's cheaper?
If you shop at an online store, do you know that it's $50 cheaper to buy 5 books for $50 and sell them to a competitor?
If you buy a book online from Amazon, you know that it costs $50 to buy 5 books for $50 and sell them to a competitor?
If you shop at a third party, do you know that it costs $50 to buy 5 books for $50 and sell them to a competitor?
I can't really say for sure, but I can say that if you do ask someone to sell you books and you ask for $50 or $50 for 5
To help you to avoid the confusion, I've included links to some good resources on how to solve step-by-step (and more on
How to Sell Less Books
You may think that

===== ROLE-BASED PROMPTING =====
You are a professional financial advisor. Explain the importance of saving money. The purpose of this essay is to explai

You are a professional financial advisor. Explain the importance of saving money. The purpose of this essay is to explai

You are a professional financial advisor and you have an understanding of the value of your financial interests. This is
```

Result:

Different prompting strategies were successfully designed and evaluated. The experiment demonstrated that few-shot and chain-of-thought prompting significantly improve response accuracy and reasoning capability compared to zero-shot prompting. It was observed that effective prompt engineering plays a critical role in maximizing the performance of Large Language Models without modifying model parameters.

Experiment 5: Fine-Tune Large Language Models Using Parameter-Efficient Techniques

Aim

To fine-tune a pretrained Large Language Model (LLM) using parameter-efficient techniques such as LoRA and adapters, and evaluate its performance on a downstream NLP task.

Procedure:

Install and import required libraries
Install Transformers, Datasets, PEFT, Accelerate, and Bitsandbytes, and import necessary modules.

Load dataset and tokenizer
Load the IMDB dataset and preprocess text using a pretrained tokenizer (BERT).

Load pretrained model
Initialize a pretrained sequence classification model (e.g., BERT) with required output labels.

Apply LoRA configuration
Define LoRA parameters and integrate them into the model to enable parameter-efficient fine-tuning.

Train the model
Configure training arguments and use the Trainer API to train the model on a subset of the dataset.

Evaluate model performance
Evaluate the trained model on test data and analyze metrics such as loss and accuracy.

Code:

```
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from peft import LoraConfig, get_peft_model
from transformers import TrainingArguments, Trainer
dataset = load_dataset("imdb")
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
def tokenize_function(example):
    return tokenizer(example["text"], truncation=True, padding="max_length")
dataset = dataset.map(tokenize_function, batched=True)
model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["query", "value"],
    lora_dropout=0.1,
    bias="none",
    task_type="SEQ_CLS"
)
model = get_peft_model(model, lora_config)
print("\n===== TRAINABLE PARAMETERS =====")
model.print_trainable_parameters()
training_args = TrainingArguments(
    output_dir="./results",
    per_device_train_batch_size=8,
```

```
    num_train_epochs=1,
    logging_dir="./logs"
)
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"].shuffle(seed=42).select(range(1000)),
    eval_dataset=dataset["test"].shuffle(seed=42).select(range(500)),
)
trainer.train()
results = trainer.evaluate()
print(results)
```

Output:

```
Map: 100% ■■■
```